

Table of Contents

1. Architecture Overview	6
1.1 System Architecture	6
1.2 Deployment Architecture	6
1.3 Multi-Tenant Data Architecture	7
2. Project Structure	7
2.1 Directory Layout	7
2.2 Key Configuration Files	8
2.3 Environment Variables	8
3. Coding Standards	9
3.1 TypeScript and React Conventions	9
3.2 Component Architecture Patterns	9
3.3 State Management Patterns (Zustand)	9
3.4 API Route Conventions	10
3.5 Database and Prisma Conventions	10
3.6 Naming Conventions	11
3.7 Error Handling Patterns	11
4. Module-Wise Code Explanation	12
4.1 Platform	12
4.1.1 Super Admin	12

4.1.2 AI Admin Console	12
4.1.3 Tenant Admin	12
4.2 Core HR	13
4.2.1 Dashboard	13
4.2.2 Company Management	13
4.2.3 Employees	13
4.3 Talent Acquisition	14
4.3.1 Recruitment	14
4.3.2 Requisitions	14
4.3.3 Offers	14
4.3.4 Job Portal	15
4.3.5 AI Interview	15
4.4 Lifecycle	15
4.4.1 Preboarding	15
4.4.2 Onboarding	16
4.4.3 Attendance	16
4.4.4 Leave	16
4.4.5 Separation	17
4.4.6 FNF	17
4.5 Compensation	17
4.5.1 Payroll	17
4.5.2 Salary Structures	18
4.6 Growth	18

4.6.1 Performance	18
4.6.2 Training	18
4.6.3 Engagement	19
4.6.4 Succession Planning	19
4.7 Operations	19
4.7.1 Projects	19
4.7.2 Timesheets	20
4.7.3 Travel	20
4.7.4 Expenses	20
4.7.5 Assets	21
4.7.6 Documents	21
4.8 External	21
4.8.1 Clients	21
4.8.2 Vendors	22
4.9 Support	22
4.9.1 Helpdesk	22
4.9.2 Grievances	22
4.9.3 AI Assistant	23
4.10 Governance	23
4.10.1 Workflows	23
4.10.2 Reports	23
4.10.3 Settings	24
4.10.4 Notifications	24

5. Design System	24
5.1 Design Tokens	24
5.2 Component Patterns	25
5.3 Sidebar Navigation Architecture	25
5.4 Form Patterns (Inline, No Modals)	26
5.5 View and Read-Only Patterns	26
6. API Architecture	27
6.1 Authentication Middleware	27
6.2 Request and Response Patterns	27
6.3 Error Handling	28
6.4 Pagination	28
6.5 Role-Based Access Control	28
7. Database Schema	29
7.1 Entity Overview	29
7.2 Entity Relationships	31
7.3 Multi-Tenant Data Isolation	32
7.4 Indexing Strategy	32
7.5 Migration Approach	32
8. Deployment and DevOps	33
8.1 Vercel Configuration	33

8.2 Environment Variables	33
8.3 Build Process	33
8.4 CI/CD Considerations	34

1. Architecture Overview

1.1 System Architecture

The NEXUS HRMS platform implements a modern, cloud-native architecture built on a decoupled frontend-backend model. The frontend, rendered by Next.js 16 with React 19, operates as a server-side rendered application deployed on the Vercel edge network. The backend leverages Next.js API routes that communicate with a Neon serverless PostgreSQL database through Prisma ORM v7 with the `@prisma/adaptor-neon` adapter for connection pooling and optimized query execution. The architecture follows a multi-tenant SaaS model where data isolation is enforced at the application layer through tenant ID propagation on every database query, ensuring complete data segregation between organizations.

The AI capabilities are powered by the `z-ai-web-dev-sdk`, which provides seamless integration for intelligent features such as AI-driven interviews, conversational HR assistants, automated resume screening, and predictive analytics. All AI operations are routed through dedicated API endpoints that manage request queuing, rate limiting, and response caching to maintain system responsiveness under load.

Frontend Framework	Next.js 16 with React 19, Server Components, and App Router
UI Library	shadcn/ui components built on Radix UI primitives with Tailwind CSS 4
State Management	Zustand for client-side state with 3 primary stores (auth, UI, notifications)
Backend Runtime	Next.js API Routes (Edge + Node.js runtimes) on Vercel
Database	PostgreSQL via Neon Serverless with WebSocket connection support
ORM	Prisma ORM v7 with <code>@prisma/adaptor-neon</code> for optimized Neon integration
Authentication	Custom JWT (jose + bcryptjs) with HS256, 24-hour expiry
AI Integration	<code>z-ai-web-dev-sdk</code> for AI interviews, chat, resume screening, and analytics
Deployment	Vercel with automatic preview deployments and production CDN
CSS Framework	Tailwind CSS 4 with custom design tokens (nexus-card, nexus-badge, etc.)

Table 1.1: NEXUS HRMS Technology Stack

1.2 Deployment Architecture

The platform is deployed on the Vercel infrastructure, which provides global edge distribution, automatic SSL, and serverless function execution. The Neon PostgreSQL database operates as a serverless instance with auto-scaling capabilities, branching for development environments, and

point-in-time recovery. The connection between the application and database uses Prisma connection pooling with the `@prisma/adapters-neon` adapter, which supports WebSocket connections for real-time data synchronization and HTTP connections for standard CRUD operations.

The deployment pipeline follows a Git-based workflow where pushes to the main branch trigger production deployments, while pull requests generate preview deployments with unique URLs. Environment variables are managed through the Vercel encrypted environment variable system with a fallback chain of `POSTGRES_PRISMA_URL`, `POSTGRES_URL`, and `DATABASE_URL` for database connectivity, ensuring resilience across different configuration scenarios.

1.3 Multi-Tenant Data Architecture

The multi-tenant architecture implements a hierarchical organizational model: Super Admin manages the entire platform, Tenant represents an organization subscribing to the service, Company Groups aggregate multiple companies, Companies contain Branches, Branches house Departments, and Departments employ Employees. This seven-level hierarchy enables fine-grained data isolation and access control while maintaining referential integrity across the organizational structure.

Data isolation is enforced at the application layer through a consistent tenant ID filter applied to all database queries via Prisma middleware. Each API route validates the requesting user tenant context before executing any database operation, preventing cross-tenant data access. The schema design ensures that every top-level entity includes a `tenantId` foreign key, and all child entities inherit the tenant context through their parent relationships. This approach provides logical data isolation without the overhead of separate database instances per tenant.

2. Project Structure

2.1 Directory Layout

The project follows Next.js 16 App Router conventions with a well-organized directory structure that separates concerns across pages, API routes, components, libraries, and configuration files. The `src` directory serves as the root for all application code, with the `app` directory containing route definitions using the file-system based routing approach. The `components` directory is organized into feature-specific subdirectories and a shared UI component library, while the `lib` directory houses utility functions, configuration, and type definitions.

<code>src/app/(dashboard)/</code>	33 page routes organized by module with nested layouts and loading states
-----------------------------------	---

src/app/api/	75+ API route files across 25+ domain directories following RESTful conventions
src/components/hrms/	32 feature-specific HRMS components for module UI rendering
src/components/ui/	48 shadcn/ui base components with Radix UI primitives
src/lib/	10 library files for utilities, validators, and configuration
src/stores/	3 Zustand stores for authentication, UI state, and notification management
prisma/	Prisma schema with 40 models, migration files, and seed scripts
public/	Static assets including images, icons, and document templates

Table 2.1: Project Directory Structure

2.2 Key Configuration Files

next.config.js	Next.js configuration with image domains, environment variables, and middleware settings
tailwind.config.ts	Tailwind CSS 4 configuration with custom design tokens and theme extensions
prisma/schema.prisma	Database schema definition with 40 models, relations, indexes, and enums
tsconfig.json	TypeScript configuration with path aliases and strict type checking enabled
package.json	Dependencies including Next.js 16, React 19, Prisma v7, Zustand, and z-ai-web-dev-sdk
.env.local	Environment variables for database URLs, JWT secrets, and API keys

2.3 Environment Variables

Environment variables follow a strict fallback chain pattern for database connectivity and configuration. The primary connection string `POSTGRES_PRISMA_URL` is used by Prisma with the Neon adapter, falling back to `POSTGRES_URL` for direct connections, and finally `DATABASE_URL` as the universal fallback. This chain ensures the application can connect across local development, preview deployments, and production environments without code changes.

POSTGRES_PRISMA_URL	Primary Neon connection string with Prisma adapter optimization
POSTGRES_URL	Fallback direct Neon connection string for non-Prisma operations
DATABASE_URL	Universal fallback connection string for local development
JWT_SECRET	Secret key for HS256 JWT token signing (minimum 32 characters)

NEXT_PUBLIC_APP_URL	Public application URL for CORS and callback configuration
Z_AI_API_KEY	API key for z-ai-web-dev-sdk AI feature integration

Table 2.3: Environment Variable Configuration

3. Coding Standards

3.1 TypeScript and React Conventions

The codebase enforces strict TypeScript configuration with no implicit any, strict null checks, and explicit return types on exported functions. All React components are authored as functional components with TypeScript interfaces defining props. Server Components are the default for all page routes and data-fetching components, while Client Components are used only when interactivity (useState, useEffect, event handlers) is required, explicitly marked with the "use client" directive.

Component props interfaces are defined inline for simple components and exported as named types for reusable components. All API responses are typed with dedicated response interfaces, and Zod schemas are used for runtime validation of form inputs and API payloads. The naming convention follows PascalCase for components and types, camelCase for functions and variables, and SCREAMING_SNAKE_CASE for constants.

3.2 Component Architecture Patterns

Components follow a strict hierarchy pattern: Page components at the route level compose Module components, which compose Feature components, which compose UI primitives from shadcn/ui. Each module follows a consistent structure with a landing page displaying icon tiles for sub-features, a context-aware sidebar with sub-menus, and embedded inline forms for data entry. The pattern explicitly prohibits modal dialogs and popup windows; all data entry and editing occurs through inline forms that expand within the content area.

The view/read-only pattern uses a viewingId state variable combined with a FiEye (eye icon) button to toggle between list and detail views. When a user clicks the view button, the component sets viewingId to the selected record ID, triggering a conditional render that displays the detail panel inline without navigation. This pattern provides a seamless, single-page experience that eliminates context switching and preserves scroll position.

3.3 State Management Patterns (Zustand)

The application uses three primary Zustand stores, each with a clear responsibility boundary. The `useAuthStore` manages authentication state including the current user, their role, tenant context, and JWT token. The `useUIStore` handles sidebar state, active module, theme preferences, and notification panel visibility. The `useNotificationStore` manages real-time notifications with unread counts and notification history.

useAuthStore	User, role, tenantId, token, permissions, login/logout actions, role-based sidebar filtering
useUIStore	Sidebar open/close, active module path, theme mode, modal state (deprecated), view mode
useNotificationStore	Notification list, unread count, mark-as-read, real-time push integration

Table 3.3: Zustand Store Definitions

Zustand stores are accessed through custom hooks that provide memoized selectors to prevent unnecessary re-renders. The auth store includes a middleware that validates the JWT token on initialization and automatically redirects to login when tokens expire. The sidebar navigation items are filtered through the `useAuthStore` based on the user role, ensuring that users only see modules and features they have permission to access.

3.4 API Route Conventions

API routes follow a consistent structure: each route file exports named HTTP method handlers (GET, POST, PUT, DELETE) that follow a middleware chain pattern. The first step in every handler is JWT token validation using the `jose` library, followed by role-based access checking, request body validation using Zod schemas, database operations through Prisma, and structured response formatting. Error handling follows a consistent pattern with try-catch blocks that return appropriate HTTP status codes and error messages.

Route files are organized by domain under `src/app/api/`, with each domain directory containing `route.ts` files for related endpoints. For example, the `employees` domain contains routes for CRUD operations, bulk import/export, and profile image upload. All routes return responses in a standardized format with success boolean, data payload, error string, and message fields, using appropriate HTTP status codes (200 for success, 201 for creation, 400 for validation errors, 401 for unauthorized, 403 for forbidden, 404 for not found, 500 for server errors).

3.5 Database and Prisma Conventions

Prisma models follow a consistent naming convention where model names use PascalCase singular form (Employee, not Employees), field names use camelCase, and enum values use SCREAMING_SNAKE_CASE. Every model that belongs to a tenant includes a `tenantId` field with a

foreign key relation to the Tenant model. Relations are defined explicitly with onDelete: Cascade for dependent records and onDelete: SetNull for optional associations.

Database queries use Prisma generated client with type-safe findMany, findUnique, create, update, and delete operations. Complex queries use include for relation loading and where with logical operators for filtering. Transactions are implemented using Prisma interactive transactions for multi-step operations that require atomicity, such as payroll processing and employee onboarding workflows. The @prisma/adaptor-neon adaptor is configured for connection pooling with WebSocket support for real-time features.

3.6 Naming Conventions

Components	PascalCase: EmployeeList.tsx, LeaveRequestForm.tsx
Hooks	camelCase with use prefix: useAuthStore, useDebounce
API Routes	kebab-case directories: /api/salary-structures/, /api/ai-interview/
Database Models	PascalCase singular: Employee, LeaveType, JobPosting
Database Fields	camelCase: firstName, tenantId, startDate
Enums	SCREAMING_SNAKE_CASE values: SUPER_ADMIN, HR_ADMIN
Constants	SCREAMING_SNAKE_CASE: JWT_EXPIRY, MAX_PAGE_SIZE
CSS Classes	nexus- prefix tokens: nexus-card, nexus-badge, nexus-border
Zustand Stores	camelCase with use prefix: useAuthStore, useUIStore
Type Interfaces	PascalCase with suffix: EmployeeResponse, LeaveRequestInput

Table 3.6: Naming Convention Reference

3.7 Error Handling Patterns

Error handling follows a layered approach: API routes use try-catch blocks with specific error type checking, Prisma errors are caught and translated to user-friendly messages, and the frontend uses React Error Boundaries for component-level error isolation. All API errors return a consistent JSON structure with success: false, an error code string, and a human-readable message. The frontend toast notification system displays errors to users with severity-appropriate styling and auto-dismiss timing.

Common Prisma error codes are mapped to HTTP status codes: P2002 (unique constraint violation) maps to 409 Conflict, P2025 (record not found) maps to 404 Not Found, and P2003 (foreign key constraint) maps to 400 Bad Request. Validation errors from Zod schemas are formatted into field-level error objects that the frontend can display inline on form fields. Network errors and timeouts trigger retry logic with exponential backoff for non-critical operations.

4. Module-Wise Code Explanation

This section provides detailed technical documentation for all 35 modules organized by their functional category. Each module description covers the route structure, component hierarchy, API endpoints, database models, state management approach, and key implementation patterns used throughout the codebase.

4.1 Platform

4.1.1 Super Admin

Platform-wide administration panel with tenant provisioning, subscription management, and system configuration. Provides oversight across all tenants with global analytics and audit capabilities.

Route Path	src/app/(dashboard)/super-admin/
API Endpoints	/api/super-admin/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Tenant, User, Subscription, SubscriptionPlan
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.1.2 AI Admin Console

Centralized AI feature management interface for configuring z-ai-web-dev-sdk integrations, managing AI interview parameters, chat bot settings, and intelligent workflow triggers.

Route Path	src/app/(dashboard)/ai-admin/
API Endpoints	/api/ai-chat, ai-interview/ (GET, POST, PUT, DELETE with role-based access)
Database Models	User, AuditLog
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.1.3 Tenant Admin

Tenant-level administration for managing company groups, companies, branches, and departments within the tenant hierarchy. Includes tenant-specific settings and user management.

Route Path	src/app/(dashboard)/tenants/
API Endpoints	/api/tenants/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Tenant, CompanyGroup, Company, Branch, Department, User
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.2 Core HR

4.2.1 Dashboard

Central HR dashboard presenting KPIs, pending actions, attendance summaries, leave balances, and recruitment pipeline status. Uses aggregated API data from multiple endpoints.

Route Path	src/app/(dashboard)/dashboard/
API Endpoints	/api/dashboard, analytics/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Employee, Attendance, LeaveBalance, LeaveRequest, JobApplication
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.2.2 Company Management

Full CRUD management for company entities including company groups, branches, and organizational structure. Supports the multi-tenant hierarchy from Company Group down to Departments.

Route Path	src/app/(dashboard)/companies/
API Endpoints	/api/companies/ (GET, POST, PUT, DELETE with role-based access)
Database Models	CompanyGroup, Company, Branch, Department, Designation
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.2.3 Employees

Comprehensive employee lifecycle management covering personal details, employment information, qualifications, experience, skills, and dependents. Central module connecting to payroll, attendance, and

performance.

Route Path	src/app/(dashboard)/employees/
API Endpoints	/api/employees/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Employee, Dependent, Qualification, Experience, EmployeeSkill, User
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.3 Talent Acquisition

4.3.1 Recruitment

End-to-end recruitment pipeline from job posting to candidate selection. Manages job postings, application tracking, interview scheduling, and candidate evaluation workflows.

Route Path	src/app/(dashboard)/recruitment/
API Endpoints	/api/recruitment, job-postings/ (GET, POST, PUT, DELETE with role-based access)
Database Models	JobPosting, JobApplication, Interview
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.3.2 Requisitions

Staff requisition management allowing departments to request new hires with approval workflows. Tracks requisition status from draft through approval to recruitment initiation.

Route Path	src/app/(dashboard)/requisitions/
API Endpoints	/api/requisitions/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Requisition, Department, Designation
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.3.3 Offers

Offer letter management with template-based generation, approval workflows, and tracking. Integrates with salary structures for compensation package configuration.

Route Path	src/app/(dashboard)/offers/
API Endpoints	/api/offers/ (GET, POST, PUT, DELETE with role-based access)
Database Models	JobApplication, SalaryStructure, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.3.4 Job Portal

Public-facing job portal for candidates to browse openings, submit applications, and track status. Supports branded career pages per tenant/company.

Route Path	src/app/(dashboard)/job-portal/
API Endpoints	/api/job-postings/ (GET, POST, PUT, DELETE with role-based access)
Database Models	JobPosting, JobApplication
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.3.5 AI Interview

AI-powered interview module using z-ai-web-dev-sdk for automated candidate screening, question generation, and preliminary evaluation. Supports configurable interview templates and scoring rubrics.

Route Path	src/app/(dashboard)/ai-interview/
API Endpoints	/api/ai-interview/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Interview, JobApplication
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.4 Lifecycle

4.4.1 Preboarding

Pre-boarding task management for new hires before their start date. Includes document collection, IT setup requests, policy acknowledgments, and welcome kit preparation.

Route Path	src/app/(dashboard)/preboarding/
API Endpoints	/api/onboarding/ (GET, POST, PUT, DELETE with role-based access)
Database Models	OnboardingTask, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.4.2 Onboarding

Structured onboarding workflow with task tracking, buddy assignment, training schedules, and milestone completion. Ensures consistent new hire experience across the organization.

Route Path	src/app/(dashboard)/onboarding/
API Endpoints	/api/onboarding/ (GET, POST, PUT, DELETE with role-based access)
Database Models	OnboardingTask, Employee, Training
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.4.3 Attendance

Real-time attendance tracking with check-in/check-out, shift management, overtime calculation, and attendance regularization. Supports biometric and geolocation integration.

Route Path	src/app/(dashboard)/attendance/
API Endpoints	/api/attendance/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Attendance, Shift, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.4.4 Leave

Comprehensive leave management with leave type configuration, balance tracking, request/approval workflows, and holiday calendar integration. Supports carry-forward and encashment rules.

Route Path	src/app/(dashboard)/leave/
API Endpoints	/api/leave/ (GET, POST, PUT, DELETE with role-based access)
Database Models	LeaveType, LeaveBalance, LeaveRequest, Holiday
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.4.5 Separation

Employee separation and offboarding management covering resignation, termination, and retirement workflows. Includes notice period tracking, asset recovery, and knowledge transfer tasks.

Route Path	src/app/(dashboard)/separation/
API Endpoints	/api/separation/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Separation, Employee, AssetAssignment
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.4.6 FNF

Full and Final settlement calculation processing exit dues, pending salary, leave encashment, recoverable advances, and final payout computation with statutory compliance.

Route Path	src/app/(dashboard)/fnf/
API Endpoints	/api/fnf/ (GET, POST, PUT, DELETE with role-based access)
Database Models	FNFCalculation, Separation, Employee, LeaveBalance
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.5 Compensation

4.5.1 Payroll

End-to-end payroll processing engine supporting multiple pay frequencies, tax calculations, statutory deductions, and bank file generation. Handles salary computation, approval workflows, and payment disbursement.

Route Path	src/app/(dashboard)/payroll/
API Endpoints	/api/payroll/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Payroll, SalaryStructure, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.5.2 Salary Structures

Configurable salary structure definitions with components for basic, allowances, deductions, and employer contributions. Supports multiple structure templates per company and designation level.

Route Path	src/app/(dashboard)/salary-structures/
API Endpoints	/api/salary-structures/ (GET, POST, PUT, DELETE with role-based access)
Database Models	SalaryStructure, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.6 Growth

4.6.1 Performance

Performance review management with configurable review cycles, rating scales, competency frameworks, and multi-rater feedback. Supports self-assessment, manager review, and calibration workflows.

Route Path	src/app/(dashboard)/performance/
API Endpoints	/api/performance/ (GET, POST, PUT, DELETE with role-based access)
Database Models	PerformanceReview, Goal, Feedback, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.6.2 Training

Training program management covering course creation, scheduling, enrollment tracking, and completion certification. Supports both internal and external training programs with budget tracking.

Route Path	src/app/(dashboard)/training/
API Endpoints	/api/training/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Training, TrainingEnrollment, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.6.3 Engagement

Employee engagement module with pulse surveys, feedback collection, and sentiment analysis. Provides engagement scores and trend analysis to help HR improve workplace satisfaction.

Route Path	src/app/(dashboard)/engagement/
API Endpoints	/api/analytics, reports/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Feedback, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.6.4 Succession Planning

Succession planning for key positions with talent pool identification, readiness assessment, and development plan tracking. Ensures business continuity for critical roles.

Route Path	src/app/(dashboard)/succession/
API Endpoints	/api/employees, performance/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Employee, PerformanceReview, Goal
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.7 Operations

4.7.1 Projects

Project management with project creation, team assignment, milestone tracking, and progress monitoring. Integrates with timesheets and resource allocation for project-based costing.

Route Path	src/app/(dashboard)/projects/
API Endpoints	/api/projects/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Project, ProjectTask, ProjectMilestone, ProjectAllocation
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.7.2 Timesheets

Time tracking module for logging work hours against projects and tasks. Supports daily, weekly, and monthly views with approval workflows and overtime calculation.

Route Path	src/app/(dashboard)/timesheets/
API Endpoints	/api/timesheets/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Timesheet, ProjectTask, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.7.3 Travel

Travel request management covering itinerary planning, approval workflows, budget allocation, and travel policy compliance. Integrates with expense claims for reimbursement.

Route Path	src/app/(dashboard)/travel/
API Endpoints	/api/travel/ (GET, POST, PUT, DELETE with role-based access)
Database Models	TravelRequest, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.7.4 Expenses

Expense claim management with receipt upload, categorization, approval workflows, and reimbursement processing. Supports per-diem rates and expense policy enforcement.

Route Path	src/app/(dashboard)/expenses/
API Endpoints	/api/expenses/ (GET, POST, PUT, DELETE with role-based access)
Database Models	ExpenseClaim, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.7.5 Assets

Asset management for IT and non-IT assets including procurement tracking, assignment, maintenance scheduling, and depreciation calculation. Supports asset lifecycle from procurement to disposal.

Route Path	src/app/(dashboard)/assets/
API Endpoints	/api/assets/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Asset, AssetAssignment, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.7.6 Documents

Document management with version control, access permissions, expiry tracking, and document templates. Supports employee document uploads, company policies, and compliance document storage.

Route Path	src/app/(dashboard)/documents/
API Endpoints	/api/documents/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Document, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.8 External

4.8.1 Clients

Client management with contact details, project associations, billing information, and communication history. Supports client onboarding and relationship management.

Route Path	src/app/(dashboard)/clients/
API Endpoints	/api/clients/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Client, Project
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.8.2 Vendors

Vendor management covering vendor registration, service catalog, contract tracking, and performance evaluation. Supports vendor onboarding and compliance documentation.

Route Path	src/app/(dashboard)/vendors/
API Endpoints	/api/vendors/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Vendor, Asset
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.9 Support

4.9.1 Helpdesk

IT and HR helpdesk with ticket categorization, priority assignment, SLA tracking, and resolution workflows. Supports knowledge base integration for self-service.

Route Path	src/app/(dashboard)/helpdesk/
API Endpoints	/api/tickets/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Ticket, TicketComment, TicketCategory
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.9.2 Grievances

Grievance management with confidential submission, investigation workflows, resolution tracking, and escalation mechanisms. Ensures fair and timely resolution of workplace concerns.

Route Path	src/app/(dashboard)/grievances/
API Endpoints	/api/grievances/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Grievance, Employee
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.9.3 AI Assistant

AI-powered HR assistant using z-ai-web-dev-sdk for natural language queries about policies, leave balances, payroll details, and company information. Provides contextual, role-based responses.

Route Path	src/app/(dashboard)/ai-assistant/
API Endpoints	/api/ai-chat/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Employee, AuditLog
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.10 Governance

4.10.1 Workflows

Visual workflow builder and engine for configuring approval chains, notification rules, and automation triggers. Supports multi-step approval with conditional branching and delegation.

Route Path	src/app/(dashboard)/workflows/
API Endpoints	/api/workflows/ (GET, POST, PUT, DELETE with role-based access)
Database Models	AuditLog, Notification
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.10.2 Reports

Comprehensive reporting engine with pre-built templates, custom report builder, scheduled report generation, and export capabilities. Supports PDF, Excel, and CSV output formats.

Route Path	src/app/(dashboard)/reports/
API Endpoints	/api/reports, analytics/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Employee, Payroll, Attendance, LeaveRequest
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.10.3 Settings

System-wide configuration management covering organization settings, policy definitions, role permissions, email templates, and integration configurations. Central control panel for all module settings.

Route Path	src/app/(dashboard)/settings/
API Endpoints	/api/tenants/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Tenant, Policy, User
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

4.10.4 Notifications

Notification management with in-app notifications, email alerts, and configurable notification preferences. Supports real-time push notifications and digest-based delivery.

Route Path	src/app/(dashboard)/notifications/
API Endpoints	/api/notifications/ (GET, POST, PUT, DELETE with role-based access)
Database Models	Notification, User
State Management	Local component state with Zustand auth context for role filtering
Component Pattern	Module landing page with icon tiles, context sidebar, inline forms, viewingId detail view

5. Design System

5.1 Design Tokens

The NEXUS HRMS design system is built on a set of custom design tokens that ensure visual consistency across all 35+ modules. These tokens are defined as Tailwind CSS utility classes with the `nexus-` prefix, providing a semantic naming system that abstracts away raw CSS values. Design tokens cover card surfaces, badges, borders, text hierarchy, and interactive states, ensuring that every component in the system maintains a cohesive visual language.

nexus-card	Card surface with rounded corners, subtle shadow, and padding. Used for all module landing tiles and content containers.
nexus-badge	Status indicator badge with rounded-full shape, colored backgrounds, and text. Used for status labels (Active, Pending, Closed).
nexus-border	Consistent border style with 1px width and muted color. Used for dividers, card edges, and section separators.
nexus-text-primary	Primary text color (#242321) for headings and body content. Highest contrast and readability.
nexus-text-secondary	Secondary text color for supporting information and descriptions. Reduced contrast for visual hierarchy.
nexus-text-muted	Muted text color (#8a877e) for timestamps, captions, and placeholder text. Lowest contrast in the text scale.

Table 5.1: Design Token Definitions

5.2 Component Patterns

All HRMS feature components follow a consistent architectural pattern that ensures predictability and maintainability. The Module Landing Page pattern displays a grid of icon tiles, each representing a sub-feature or action within the module. Clicking a tile navigates to the corresponding sub-module page or triggers an inline form expansion. This pattern provides an at-a-glance overview of module capabilities while maintaining a clean, uncluttered interface.

The Inline Form pattern is a core design principle that replaces traditional modal dialogs with embedded forms that expand within the content area. When a user initiates a create or edit action, the form slides open inline, pushing existing content downward. This approach preserves spatial context, eliminates jarring modal popups, and provides a smoother user experience. Form submission triggers an optimistic update to the UI with a background API call, and the form collapses on success or displays validation errors inline.

5.3 Sidebar Navigation Architecture

The sidebar navigation implements an 11-section structure that organizes all 35+ modules into logical groupings. Each section is collapsible and displays its contained modules as navigation items. The active module is highlighted with the accent color, and the sidebar automatically expands the section

containing the current route. The sidebar items are dynamically filtered based on the user role through the useAuthStore, ensuring that users only see navigation options they have permission to access.

Section	Modules
Platform	Super Admin, AI Admin Console, Tenant Admin
Core HR	Dashboard, Company Management, Employees
Talent Acquisition	Recruitment, Requisitions, Offers, Job Portal, AI Interview
Onboarding and Lifecycle	Preboarding, Onboarding, Attendance, Leave, Separation, FNF
Time and Attendance	Attendance, Leave, Timesheets
Compensation	Payroll, Salary Structures
Performance and Growth	Performance, Training, Engagement, Succession Planning
Operations	Projects, Timesheets, Travel, Expenses, Assets, Documents
External Relations	Clients, Vendors
Support and Intelligence	Helpdesk, Grievances, AI Assistant
Governance	Workflows, Reports, Settings, Notifications

Table 5.3: Sidebar Navigation Sections

5.4 Form Patterns (Inline, No Modals)

The embedded inline form system is the primary data entry mechanism across the platform. Forms are rendered inline within the content area using a state-driven pattern: when `isFormOpen` is true, the form component renders in place, and when false, it collapses. Each form maintains its own local state using React `useState` hooks, with Zod schema validation applied on submit. Error messages appear directly below the relevant form fields, providing immediate feedback without disrupting the user workflow.

Multi-step forms use a `stepIndex` state variable to track progression, rendering the appropriate step component at each stage. Navigation between steps includes validation of the current step before allowing progression, and a step indicator shows the user position in the workflow. This pattern is used extensively in complex workflows such as employee onboarding, payroll setup, and recruitment pipeline management.

5.5 View and Read-Only Patterns

The view pattern uses a `viewingId` state variable to toggle between list and detail views without page navigation. When a user clicks the FiEye icon button next to a list item, the component sets `viewingId` to

that record ID, triggering a conditional render that displays a detail panel with all record fields in a read-only format. The detail panel includes action buttons for editing (which switches to inline edit mode) and closing (which resets viewingId to null and returns to the list view).

This pattern provides significant performance benefits over traditional page navigation, as the list data remains in memory and the detail view renders without additional data fetching for previously loaded records. For records that have not been fetched, the detail view triggers a single API call to load the complete record with all relations, displaying a loading skeleton during the fetch.

6. API Architecture

6.1 Authentication Middleware

Every API route implements a consistent authentication middleware pattern using the jose library for JWT verification. The middleware extracts the nexus_token from the Authorization header (Bearer token format), verifies the token signature using the HS256 algorithm with the configured JWT_SECRET, and decodes the payload to extract the user ID, role, and tenant ID. If the token is missing, expired, or invalid, the middleware returns a 401 Unauthorized response with a structured error message.

The JWT token is stored in localStorage under the key nexus_token and is automatically included in all API requests through an Axios interceptor configured in the API client library. Token expiry is set to 24 hours, and the application implements a silent refresh mechanism that checks token validity on application load and redirects to the login page when the token is within 5 minutes of expiry. The bcryptjs library with 12 salt rounds is used for password hashing during registration and verification during login.

6.2 Request and Response Patterns

All API endpoints follow a standardized request/response pattern that ensures consistency and predictability. Request bodies are validated using Zod schemas before processing, and response payloads follow a uniform structure. Successful responses include a success: true flag along with the data payload, while error responses include success: false with an error code and human-readable message. Pagination is implemented using cursor-based pagination for large datasets and offset-based pagination for smaller collections.

Success Response	{ success: true, data: T, message?: string }
Error Response	{ success: false, error: string, message: string }

Paginated Response	<code>{ success: true, data: T[], pagination: { page, limit, total, totalPages } }</code>
Validation Error	<code>{ success: false, error: 'VALIDATION_ERROR', details: FieldError[] }</code>

Table 6.2: API Response Format Standards

6.3 Error Handling

The API error handling system uses a layered approach that translates technical errors into user-friendly messages while preserving debugging information in server logs. At the outermost layer, a global error handler catches unhandled exceptions and returns a generic 500 Internal Server Error response. Within each route handler, specific error types are caught and mapped to appropriate HTTP status codes and error messages.

Prisma-specific errors are handled with dedicated mapping: P2002 (unique constraint) returns 409 Conflict with a message indicating which field caused the violation. P2025 (record not found) returns 404 Not Found. P2003 (foreign key constraint) returns 400 Bad Request with context about the invalid reference. P2014 (required relation violation) returns 400 Bad Request. All errors are logged with request context including the endpoint, user ID, and request parameters for debugging and audit purposes.

6.4 Pagination

List endpoints implement pagination with a consistent interface: page (default 1), limit (default 20, max 100), and optional sort and filter parameters. The response includes a pagination object with the current page, items per page, total count, and total pages. For APIs that return large datasets such as employee lists and attendance records, cursor-based pagination is available using a cursor parameter that takes the ID of the last item from the previous page, providing more consistent performance on large tables.

6.5 Role-Based Access Control

The API implements a five-tier database-level role system (super_admin, tenant_admin, hr_admin, manager, employee) with 14 application-level roles derived from these base roles. Each API route defines its required minimum role level, and the authentication middleware verifies the user role before allowing access. Super admin routes are isolated in the /api/super-admin/ directory with additional tenant-isolation checks.

Role-based access is implemented through a checkPermissions utility function that takes the required role and the user current role, returning a boolean or throwing a 403 Forbidden error. The system also supports resource-level permissions where a manager can only access data within their department, and an employee can only access their own records. These granular permissions are enforced at the Prisma query level using where clauses that filter by department or employee ID based on the user role and

position.

7. Database Schema

7.1 Entity Overview

The database schema consists of 40 Prisma models organized into functional domains. The core organizational hierarchy (Tenant, CompanyGroup, Company, Branch, Department) provides the structural foundation, while the User and Employee models serve as the central entities connecting to most other models. The schema uses PostgreSQL-specific features including enum types, JSON fields for flexible data storage, and timestamp columns with automatic management through Prisma `@updatedAt` and `@createdAt` annotations.

Model	Description
Tenant	Organization tenant with multi-tenant isolation key, subscription reference, and configuration settings
CompanyGroup	Logical grouping of companies within a tenant for consolidated reporting
Company	Individual company entity with branding, location, and regulatory details
Branch	Physical office location with address and regional settings
Department	Functional department within a branch with head designation
Designation	Job title definition with grade level and salary band reference
User	Authentication entity with email, password hash, role, and tenant association
Employee	Core employee record linking personal info, employment details, and user account
Dependent	Employee dependent information for insurance and emergency contact purposes
Qualification	Employee educational qualifications with institution and year details
Experience	Employee prior work experience with company, role, and duration
EmployeeSkill	Employee skill mapping with proficiency level and certification status
JobPosting	Published job opening with requirements, compensation range, and status
JobApplication	Candidate application against a job posting with status tracking
Interview	Interview scheduling with type, panel, feedback, and scoring
OnboardingTask	Checklist item for new hire onboarding with assignee and due date
LeaveType	Leave category definition with quota, carry-forward, and encashment rules

Model	Description
LeaveBalance	Employee leave balance per type with opening, used, and remaining counts
LeaveRequest	Leave application with dates, type, reason, and approval status
Attendance	Daily attendance record with check-in/out times and status

Table 7.1: Database Models (page 1)

Model	Description
Payroll	Monthly payroll run with gross, deductions, net pay, and processing status
SalaryStructure	Compensation template with component breakdown for basic, allowances, deductions
PerformanceReview	Review cycle entry with ratings, comments, and review status
Goal	Individual or team goal with metrics, target, and progress tracking
Feedback	Multi-rater feedback with competency ratings and qualitative comments
Training	Training program definition with syllabus, schedule, and capacity
TrainingEnrollment	Employee enrollment in a training program with completion status
Asset	Company asset with category, value, depreciation, and lifecycle status
AssetAssignment	Asset-to-employee assignment with issue and return dates
Document	Document record with type, version, access level, and expiry tracking
IncidentReport	Workplace incident with severity, investigation status, and resolution
TravelRequest	Travel request with itinerary, purpose, budget, and approval chain
ExpenseClaim	Expense claim with line items, receipts, and reimbursement status
Timesheet	Daily time entry with project, task, hours, and approval status
Promotion	Employee promotion with old/new designation, effective date, and approval
Grievance	Confidential grievance with category, priority, and resolution tracking
Separation	Employee separation with type, notice period, and clearance checklist
FNFCalculation	Full and final settlement with dues, recoveries, and net payout
Notification	System notification with type, read status, and delivery channel
AuditLog	System audit trail with entity, action, old/new values, and timestamp

Table 7.1: Database Models (page 2)

Model	Description
LoginActivity	User login tracking with IP, device, and session details
Policy	Company policy definition with category, content, and version
Holiday	Holiday calendar entry with date, type, and applicability
Announcement	Company announcement with priority, audience, and expiry
Shift	Work shift definition with timing, break rules, and applicable days
Reimbursement	Reimbursement claim with type, amount, and approval workflow
SubscriptionPlan	SaaS subscription plan with features, limits, and pricing
Subscription	Tenant subscription with plan, billing cycle, and status
Client	Client entity with contact, industry, and billing information
Vendor	Vendor entity with services, contract terms, and performance rating
Project	Project with timeline, budget, client association, and status
ProjectTask	Task within a project with assignee, estimate, and progress
ProjectMilestone	Project milestone with target date, deliverables, and completion
ProjectAllocation	Employee allocation to project with percentage and date range
TicketCategory	Helpdesk ticket category with SLA and escalation rules
Ticket	Support ticket with priority, assignee, and resolution tracking
TicketComment	Comment on a support ticket with attachments and internal flag
Requisition	Staff requisition with justification, approval chain, and fulfillment status

Table 7.1: Database Models (page 3)

7.2 Entity Relationships

The entity relationship model follows the organizational hierarchy as its primary axis. The Tenant model sits at the top of the hierarchy, with CompanyGroup, Subscription, and AuditLog as direct children. CompanyGroup contains Companies, which contain Branches, which contain Departments. The Employee model is the most connected entity in the schema, with foreign key relations to Department, User, and numerous functional entities including LeaveBalance, Attendance, Payroll, PerformanceReview, AssetAssignment, and many more.

The JobPosting and JobApplication models form the recruitment sub-graph, with JobApplication linking to Interview for the interview pipeline. The Project model connects to Client, ProjectTask, ProjectMilestone, and ProjectAllocation for project management. The Ticket model forms a self-contained helpdesk sub-graph with TicketCategory and TicketComment. All relations use explicit

onDelete and onUpdate behaviors to maintain referential integrity, with Cascade for parent-child relationships and SetNull for optional associations.

7.3 Multi-Tenant Data Isolation

Multi-tenant data isolation is enforced through a tenantId field on all top-level entities and inherited tenant context through parent relations on child entities. Prisma middleware intercepts all database queries and automatically injects the tenantId filter based on the authenticated user tenant context. This application-level isolation ensures that no query can access data belonging to another tenant, even if the query is manually constructed.

The isolation strategy uses a shared database approach rather than separate schemas or databases per tenant, which provides cost efficiency and simplified maintenance. The Tenant model includes configuration fields that control feature availability, data retention policies, and resource limits per tenant. Cross-tenant operations are exclusively available to the super_admin role through dedicated endpoints in the /api/super-admin/ route directory, which bypass the tenant filter with explicit authorization checks.

7.4 Indexing Strategy

The indexing strategy is designed to optimize the most common query patterns in the HRMS application. Every model with a tenantId field has a composite index on (tenantId, id) to support efficient tenant-scoped queries. Foreign key fields are indexed individually to accelerate join operations. The Employee model has additional indexes on email, employeeId, and departmentId to support the frequent lookup patterns in the application.

Compound indexes are defined for frequently filtered combinations: LeaveRequest has an index on (tenantId, status, startDate), Attendance on (tenantId, employeeId, date), and Payroll on (tenantId, employeeId, month, year). Full-text search is supported on JobPosting and Employee models using PostgreSQL tsvector with GIN indexes on the title and description fields. The Prisma schema defines these indexes using the @@index annotation with explicit column lists.

7.5 Migration Approach

Database migrations are managed through the Prisma migration system with a strict workflow: schema changes are made in schema.prisma, migrations are generated using prisma migrate dev during development, and applied using prisma migrate deploy in production. Each migration creates a timestamped SQL file in the prisma/migrations/ directory, which is version-controlled alongside the application code.

The migration strategy follows a zero-downtime approach where destructive changes (column removals, type changes) are executed in multiple steps: first add the new column, then migrate data, then update the application to use the new column, and finally remove the old column in a subsequent migration. For Neon-specific features, the schema leverages Neon branching capability to test migrations on a database branch before applying them to production, ensuring that migration issues are caught in a safe environment.

8. Deployment and DevOps

8.1 Vercel Configuration

The application is deployed on Vercel with a configuration optimized for Next.js 16 serverless functions and edge runtime. The `vercel.json` configuration specifies function execution regions for latency optimization, sets maximum function duration for long-running operations like payroll processing, and configures routing rules for API endpoints. The deployment automatically handles server-side rendering, static optimization, and incremental static regeneration based on page-level data requirements.

The Vercel edge network provides global CDN distribution for static assets and server-rendered pages, with automatic compression (Brotli and Gzip), image optimization through `next/image`, and intelligent caching based on cache-control headers. Preview deployments are created automatically for every pull request, enabling code review with live, shareable URLs before merging to the main branch.

8.2 Environment Variables

Environment variables are managed through the Vercel encrypted environment variable system with separate configurations for Production, Preview, and Development environments. The database connection follows a strict fallback chain: `POSTGRES_PRISMA_URL` is the primary connection string optimized for the Prisma Neon adapter with pooled connections, `POSTGRES_URL` serves as the fallback for direct Neon connections, and `DATABASE_URL` provides the universal fallback for local development and non-Neon environments.

Sensitive variables including `JWT_SECRET`, database URLs, and API keys are marked as encrypted in Vercel and are never exposed to the client bundle. Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser and are limited to non-sensitive configuration such as the application URL and feature flags. The application validates the presence of all required environment variables at startup and fails fast with descriptive error messages if any are missing.

8.3 Build Process

The build process follows the Next.js 16 optimized build pipeline, which analyzes each page and API route to determine the optimal rendering strategy (static, server-side, or edge). The build output includes a detailed analysis of bundle sizes, route configurations, and data requirements. Prisma generates its client during the postinstall hook, ensuring the latest schema is reflected in the generated types and query methods.

The build configuration includes TypeScript compilation with strict mode, ESLint checks with custom rules for React hooks and accessibility, and Tailwind CSS purging to minimize the CSS bundle. Production builds are optimized through Next.js automatic code splitting, tree shaking, and dead code elimination. The average build time is under 2 minutes, with incremental builds using the Vercel remote cache achieving sub-30-second rebuilds for typical changes.

8.4 CI/CD Considerations

The CI/CD pipeline is configured through the Vercel Git integration with additional quality gates enforced through GitHub Actions. The pipeline executes the following stages on every pull request: TypeScript type checking (`tsc --noEmit`), ESLint validation with zero-warning policy, unit test execution with Jest and React Testing Library, Prisma schema validation, and database migration dry-run against a Neon development branch.

Production deployments are triggered automatically on merges to the main branch after all quality gates pass. The deployment includes a health check phase that verifies the application responds correctly before traffic is routed to the new deployment. Rollback is achieved through the Vercel instant rollback feature that reverts to the previous deployment with a single click. Database migrations are applied as a pre-deployment step using `prisma migrate deploy`, ensuring schema changes are applied before the new code is active.